

System Architecture Specification

Melanoma Clinical Platform

Document Version: 1.2
Date: December 2025
Prepared By: Engineering Team

Document Revision History

Date	Version	Description	Author
Dec 2025	1.2	Initial draft of system architecture and API specifications.	Eng. Team

Contents

1	Overview	2
1.1	Purpose	2
1.2	System Components	2
2	Backend Architecture	3
2.1	API Specifications	3
2.2	Machine Learning Pipeline	3
2.2.1	Image Preprocessing	3
2.2.2	Inference Ensemble Strategy	3
3	Frontend Architecture	4
3.1	Framework and Configuration	4
4	Data Management and Security	5
4.1	Storage Implementations	5
4.2	Security Protocol	5

1. Overview

1.1 Purpose

This document outlines the system architecture and technical specifications of the Melanoma Clinical Platform. The system provides an automated pipeline for skin lesion analysis, integrating computer vision inference, secure storage, and active learning feedback mechanisms.

1.2 System Components

The platform is deployed as a distributed client-server application consisting of three primary layers:

- **Client Application (Frontend):** A Next.js (React) application serving as the primary user interface.
- **Application Programming Interface (Backend):** A FastAPI service handling request routing, data processing, and model inference.
- **Storage Layer:** A decentralized configuration utilizing a Dropbox cluster for large binary files (model weights) and an SQL database for transactional application data.

2. Backend Architecture

2.1 API Specifications

The backend application is developed in Python using the FastAPI framework. The RESTful API exposes the following primary endpoints:

POST /api/scan	Accepts an image payload. The system processes the image through the preprocessing pipeline, executes the inference ensemble, stores the result in the SQL database, and computes temporal growth metrics against the patient's historical records.
POST /api/feedback	Receives confirmed diagnostic labels. The payload and image are synchronized to the storage cluster for subsequent active learning cycles.
POST /api/retrain	Initiates a retraining sequence for the secondary feedback model using the accumulated dataset.
GET /api/history	Returns paginated historical scan data for a specified user identifier.

2.2 Machine Learning Pipeline

2.2.1 Image Preprocessing

Prior to inference, the system applies a preprocessing sequence designed to mitigate the effect of hair artifacts on the lesion images:

1. Conversion from RGB to LAB color space.
2. Contrast Limited Adaptive Histogram Equalization (CLAHE) applied to the L-channel.
3. Morphological Blackhat filtering for artifact isolation.
4. Telea Inpainting algorithm for structural interpolation.

2.2.2 Inference Ensemble Strategy

The platform employs a dual-model approach using selective ensembling:

- **Primary Ensemble:** Utilizes `coatnet_0_rw_224` architectures. Models are dynamically fetched from the storage cluster, constrained by a minimum AUC threshold of 0.95.
- **Secondary Feedback Model:** A `mobilenetv3_small_050` architecture trained locally on user-submitted corrections.
- **Scoring Function:** The final classification probability is computed via a weighted average. The primary ensemble contributes 20% to the final decision, while the secondary feedback model contributes 80% (`FUSION_WEIGHT = 0.80`), ensuring that recent empirical corrections are prioritized.

3. Frontend Architecture

3.1 Framework and Configuration

The client interface is a Next.js application leveraging the App Router paradigm.

- **Language:** TypeScript is strictly enforced for compile-time safety.
- **Styling:** Tailwind CSS provides utility-first styling, integrated via PostCSS.
- **Routing Structure:** The application includes dedicated routes for authentication (`/login`), core functionality (`/scans`), and billing (`/payments`).

4. Data Management and Security

4.1 Storage Implementations

- **Relational Database:** SQLAlchemy ORM interfaces with the primary SQL data store, managing user sessions, scan metadata, and historical records.
- **Cluster Storage:** The `dropbox_manager.py` module abstracts connections to a distributed array of Dropbox accounts. It manages the synchronization of the `feedback_log.csv` and serialized model weights (`.pth` files).

4.2 Security Protocol

- **Environment Variables:** Operational credentials, including Dropbox App Keys and OAuth refresh tokens, must be provisioned via environment variables prior to production deployment.
- **Cross-Origin Resource Sharing (CORS):** Current development configurations permit unrestricted cross-origin requests. Production deployment requires strict domain whitelisting.